ANDROID PRACTICE PROJECT REPORT

CryptoApp: Crash Post-Mortem & Architecture Documentation

Complete technical diagnosis of the Dagger Hilt runtime exception, compiler bytecode injection, and full resolution.

1. Executive Summary & Incident Recurrence

When running **CryptoApp** on an Android device or emulator, the application crashed immediately upon navigating to screens that invoke `hiltViewModel()`:

> **FATAL EXCEPTION: main**
> `java.lang.IllegalStateException: Given component holder class com.example.cryptoapp.MainActivity does not implement interface dagger.hilt.internal.GeneratedComponent or interface dagger.hilt.internal.GeneratedComponentManager`
> at dagger.hilt.EntryPoints.get(EntryPoints.java:62)
> at dagger.hilt.android.internal.lifecycle.HiltViewModelFactory.createInternal(HiltViewModelFactory.java:207)
> at androidx.hilt.lifecycle.viewmodel.compose.HiltViewModelKt.rememberHiltViewModelFactory(HiltViewModel.kt:99)
> at com.example.cryptoapp.AppNavigationKt.AppNavigation(AppNavigation.kt:32)

2. Deep-Dive: Why Did It Crash Even With @AndroidEntryPoint?

The most confusing aspect of this bug for Android developers is that **the annotation was present in source code**, yet Hilt insisted at runtime that `MainActivity` did not implement `GeneratedComponentManager`. Here is what actually happened:

> **The "Silent Compiler" Trap:**
> In Android, annotations like `@AndroidEntryPoint` and `@HiltAndroidApp` are just source markers. For them to do anything, **two build-time tools must be active:**
>
> 1. **The Hilt Annotation Processor / KSP Compiler:** Generates `Hilt_MainActivity.java` and Dagger graph factories.
> 2. **The Hilt Gradle Plugin (ASM Bytecode Transformer):** Rewrites `MainActivity.class` bytecode so that it extends `Hilt_MainActivity` instead of `ComponentActivity`.

In this project, `implementation("com.google.dagger:hilt-android:2.51.1")` was declared in `dependencies`, but:

- ❌ `id("com.google.dagger.hilt.android")` was **not applied** in the root or app `build.gradle.kts`.
- ❌ `id("com.google.devtools.ksp")` was **not applied**.
- ❌ `ksp("com.google.dagger:hilt-compiler:...")` was **missing** from dependencies.

Consequently, the compiler compiled `MainActivity.kt` as a plain Android Activity without running code generation or bytecode transformation. Hilt looked for `GeneratedComponentManager` at runtime, found nothing, and crashed.

```
[MainActivity.kt with @AndroidEntryPoint] │ ├── (Without Hilt Gradle Plugin & KSP): │ └── Compiles as plain ComponentActivity. ❌ ZERO Hilt code generated. │ └── Runtime: Crash (MainActivity doesn't implement GeneratedComponentManager) │ └── (With Hilt Gradle Plugin & KSP): ├── KSP generates Hilt_MainActivity.java & Dagger components. ✅ ├── ASM transforms MainActivity to inherit Hilt_MainActivity. ✅ └── Runtime: Success! GeneratedComponentManager is present. 🎉
```

3. The Complete Resolution & Build Toolchain

> **Applied Fix Summary:**
>
> 1. Upgraded Dagger Hilt to **v2.60.1** for full compatibility with Android Gradle Plugin 9.3.3.
> 2. Configured **KSP v2.2.10-2.0.2** to process annotations for Kotlin 2.2.10.
> 3. Configured `gradle.properties` with `android.newDsl=false` and `android.disallowKotlinSourceSets=false` to support KSP generated source sets in AGP 9.
> 4. Verified that KSP and Hilt tasks successfully ran: `:app:kspDebugKotlin`, `:app:hiltCollectClassesDebug`, and `:app:transformDebugClassesWithAsm`.

File 1: Root build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.compose) apply false
    id("com.google.dagger.hilt.android") version "2.60.1" apply false
    id("com.google.devtools.ksp") version "2.2.10-2.0.2" apply false
}
```

File 2: app/build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
    id("com.google.dagger.hilt.android")
    id("com.google.devtools.ksp")
}

dependencies {
    // ... other dependencies ...
    implementation("com.google.dagger:hilt-android:2.60.1")
    ksp("com.google.dagger:hilt-compiler:2.60.1")
    implementation("androidx.hilt:hilt-navigation-compose:1.4.0")
}
```

File 3: gradle.properties

```
# AGP 9.x compatibility flags for Hilt and KSP
android.newDsl=false
android.disallowKotlinSourceSets=false
android.sync.suppressAgpWarnings=UNSUPPORTED_PROJECT_OPTION_USE
```

File 4: Application & Activity Setup

```
// CoinApplication.kt
@HiltAndroidApp
class CoinApplication : Application()

// MainActivity.kt
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent { CryptoAppTheme { AppNavigation() } }
    }
}
```

4. Build & Verification Evidence

Following the toolchain configuration, Gradle executed all 41 build tasks cleanly:

Task Name	Purpose	Result
`:app:kspDebugKotlin`	Processes Hilt annotations and creates source bindings	**SUCCESS** (Generated `Hilt_MainActivity.java`)
`:app:hiltCollectClassesDebug`	Discovers all injected ViewModels and Modules	**SUCCESS** (Aggregated Dagger dependency graph)
`:app:transformDebugClassesWithAsm`	Rewrites `MainActivity.class` bytecode	**SUCCESS** (Injected `GeneratedComponentManager`)
`:app:assembleDebug`	Compiles and packages debug APK	**BUILD SUCCESSFUL** (41 actionable tasks executed)

Generated Artifacts in app/build/generated/

- `Hilt_MainActivity.java` - The generated component manager interface for `MainActivity`
- `Hilt_CoinApplication.java` - Base application class with singleton component holder
- `CoinApplication_HiltComponents.java` & `DaggerCoinApplication_HiltComponents_SingletonC.java` - Root dependency injection container
- `CoinListViewModel_HiltModules.java` & `CoinDetailViewModel_HiltModules.java` - ViewModel scoped injection providers

5. Key Architectural Takeaways

1. **Library Dependency ≠ Plugin:** Declaring `hilt-android` in `dependencies` only imports the annotations and runtime interfaces. The `com.google.dagger.hilt.android` plugin and `ksp(" … hilt-compiler … ")` are mandatory for code generation.
2. **Bleeding Edge AGP Compatibility:** When using latest Android Gradle Plugin (AGP 9.x), older Hilt versions (2.51) fail with `Android BaseExtension not found`. Hilt 2.60.1+ and KSP with `android.newDsl=false` and `android.disallowKotlinSourceSets=false` are required.
3. **Always Verify Generated Sources:** If a Hilt error mentions missing `GeneratedComponent`, check `app/build/generated/`. If no `Hilt_MainActivity.java` exists, the problem is in the Gradle plugin configuration.

CryptoApp Practice Project • Android Jetpack Compose & Hilt Updated: November 2026